

Sec 3: 転倒数ベースGreedy（符号なし順列）

対象問題：SbR, SbT, SbRT（符号なし）

前提定義

順列と単位順列

長さ n の符号なし順列 $\pi = (\pi_1 \pi_2 \dots \pi_n)$ は $\{1, 2, \dots, n\}$ の並び替えである。単位順列 $\iota = (1 \ 2 \ \dots \ n)$ がソートの目標となる。

再配置の種類と長さ

逆位 (reversal) $\rho(i, j)$ ($1 \leq i < j \leq n$) は区間 $[i, j]$ の要素を逆順にする：

$$(\pi_1 \dots \pi_{i-1} \pi_i \dots \pi_j \pi_{j+1} \dots \pi_n) \cdot \rho(i, j) = (\pi_1 \dots \pi_{i-1} \pi_j \dots \pi_i \pi_{j+1} \dots \pi_n)$$

転置 (transposition) $\tau(i, j, k)$ ($1 \leq i < j < k \leq n+1$) は隣接する2つの区間 $[i, j]$ と $[j, k]$ を交換する：

$$(\pi_1 \dots \pi_{i-1} \pi_i \dots \pi_{j-1} \pi_j \dots \pi_{k-1} \pi_k \dots \pi_n) \cdot \tau(i, j, k) = (\pi_1 \dots \pi_{i-1} \pi_j \dots \pi_{k-1} \pi_i \dots \pi_{j-1} \pi_k \dots \pi_n)$$

再配置 β の長さ $|\beta|$ を影響を受ける要素数で定義する：

- $|\rho(i, j)| = j - i + 1$
- $|\tau(i, j, k)| = k - i$

整数 $\lambda > 1$ に対し、 $|\beta| \leq \lambda$ を満たす再配置を **λ -再配置** と呼ぶ。再配置モデル M はソートに使用できる λ -再配置の集合（逆位のみ・転置のみ・両方など）を指す。

コスト関数

再配置 β のコストを **$f(\beta) = |\beta|$** と定義する（本節では $\alpha = 1$ ）。操作列 $S = \beta_1, \beta_2, \dots, \beta_m$ の総コストは $f(S) = \sum f(\beta_i)$ 。

λ -再配置距離

順列 π とモデル M に対する **λ -再配置距離** $c^M_\lambda(\pi)$ は、 M の λ -再配置のみを使って π を ι にソートする操作列の最小総コストである。本節では以下の記法を使う：

- $c^\lambda(\pi)$: SbR（符号なし λ -逆位のみ）の距離
- $c^t_\lambda(\pi)$: SbT（ λ -転置のみ）の距離
- $c^{rt}_\lambda(\pi)$: SbRT（ λ -逆位と λ -転置）の距離

転倒数

要素の対 (π_i, π_j) が **転倒 (inversion)** であるとは、 $i < j$ かつ $\pi_i > \pi_j$ が成り立つことをいう。転倒数 $\text{Inv}(\pi)$ を π に含まれる転倒の総数として定義する：

$$\text{Inv}(\pi) = |\{(i, j) : i < j, \pi_i > \pi_j\}|$$

基本性質：

- $\text{Inv}(\pi) = 0 \iff \pi = \iota$ （符号なし順列においては単位順列のみ）
- $\text{Inv}(\pi) > 0$ ならば隣接転倒 (π_i, π_{i+1}) が必ず存在する（Lemma 1）

例： $\pi = (3 \ 4 \ 1 \ 2)$ の転倒は $(3, 1), (3, 2), (4, 1), (4, 2)$ の4つなので $\text{Inv}(\pi) = 4$ 。

また、 π と再配置 β に対し $\Delta \text{Inv}(\pi, \beta) = \text{Inv}(\pi) - \text{Inv}(\pi \cdot \beta)$ を β による転倒数の削減量とする。

アルゴリズム (Algorithm 1)

コスト1あたりに最も多くの転倒を除去できる λ -再配置を毎ステップ貪欲に選んで適用する。

```
入力：符号なし順列  $\pi$ 、整数  $\lambda > 1$ 、 $\lambda$ -再配置のみからなるモデル  $M$ 
出力： $\pi$  をソートする  $M$  の操作列

while  $\text{Inv}(\pi) > 0$ :
     $\beta = \text{argmax}\{ \Delta \text{Inv}(\pi, \beta) / f(\beta) : \beta \in M \}$  ← コスト効率最大の操作を選ぶ
     $\pi \leftarrow \pi \cdot \beta$ 
     $S$  に  $\beta$  を追加
return  $S$ 
```

$\text{Inv}(\pi) = 0 \iff \pi = \iota$ (符号なし) なので、転倒数が0になれば終了する。Lemma 5 より常に転倒数を減らす操作が存在するため、アルゴリズムは必ず終了する。

鍵となる補題

Lemma 1 (隣接転倒の存在) $\text{Inv}(\pi) > 0$ ならば、隣接転倒 (π_i, π_{i+1}) が必ず存在する。

証明の概略： $|\pi_1| < \dots < |\pi_i|$ となる最長の先頭部分列を考えると、 $\text{Inv}(\pi) > 0$ より $i < n$ であり、 (π_i, π_{i+1}) が転倒になる。

Lemma 3 (各操作の比の上界)

任意の順列 π と整数 λ に対して：

- λ -逆位 ρ : $\Delta \text{Inv}(\pi, \rho) / |\rho| \leq (\lambda-1)/2$
- λ -転置 τ : $\Delta \text{Inv}(\pi, \tau) / |\tau| \leq \lambda/4$

証明の概略：長さ k の逆位が除去できる転倒は区間内の $k(k-1)/2$ 個以下なので比 $\leq (k-1)/2$ 。転置 $\tau(i,j,k)$ は区間 $[i,j]$ と $[j,k]$ をまたぐ転倒のみ除去できるため、その数は $(j-i)(k-j) \leq (k/2)^2$ となり比 $\leq k/4$ 。

Lemma 4 (距離の下界)

任意の符号なし順列 π と $\lambda > 1$ に対して：

- $c^r \lambda(\pi) \geq c^t \lambda(\pi) \geq 2 \cdot \text{Inv}(\pi) / (\lambda-1)$
- $c^t \lambda(\pi) \geq 4 \cdot \text{Inv}(\pi) / \lambda$

証明の概略：Lemma 3 より、転倒数を1減らす最小コストは逆位で $2/(\lambda-1)$ 、転置で $4/\lambda$ 。 $\text{Inv}(\pi)$ 個の転倒をすべて解消するには少なくともその積のコストが必要。

Lemma 5 (アルゴリズムのコスト上界)

任意の符号なし順列 π と $\lambda > 1$ に対して：

- $c^t \lambda(\pi) \leq 2 \cdot \text{Inv}(\pi)$
- $c^r \lambda(\pi) \leq c^r \lambda(\pi) \leq 2 \cdot \text{Inv}(\pi)$

証明の概略：Lemma 1 より隣接転倒 (π_i, π_{i+1}) が存在するとき、 $\tau(i, i+1, i+2)$ と $\rho(i, i+1)$ はいずれもコスト2でその転倒を1つ除去する。よって「コスト2で転倒1削減」が常に可能であり、 $\text{Inv}(\pi)$ 個の転倒を解消する総コスト

は $2 \cdot \ln v(\pi)$ 以下。

主要定理

Theorem 1 : $\lambda > 1$ のとき、Algorithm 1 は

- SbR と SbRT に対して： **$(\lambda-1)$ -近似**
- SbT に対して： **$(\lambda/2)$ -近似**

証明の流れ：

	SbR / SbRT	SbT
アルゴリズムのコスト上界 (Lemma 5)	$\leq 2 \cdot \ln v(\pi)$	$\leq 2 \cdot \ln v(\pi)$
最適コストの下界 (Lemma 4)	$\geq 2 \cdot \ln v(\pi) / (\lambda - 1)$	$\geq 4 \cdot \ln v(\pi) / \lambda$
近似係数	$2 \cdot \ln v(\pi) / [2 \cdot \ln v(\pi) / (\lambda - 1)] = \lambda - 1$	$2 \cdot \ln v(\pi) / [4 \cdot \ln v(\pi) / \lambda] = \lambda / 2$